



Mapeamento de Elementos HTML - Divvino

Objetivo

Aplicar diferentes estratégias de mapeamento de elementos HTML em uma página real da loja **Divvino**, visando à automação de testes. Foram priorizadas abordagens que privilegiam a manutenção dos testes, como seletores semânticos, hierárquicos e uso de atributos estáveis.

Site: <https://www.divvino.com.br/>

Elementos Mapeados

1º Elemento: Array de Botões de Categorias



- Nome do elemento: Link de uma categoria no menu superior.
- HTML Relevante:

```
<ul class="nav-links">  
  <li class="nav-item">  
    <a class="nav-link" href="/vinhos">Vinhos</a>  
  </li>  
</ul>
```

- ELEMENT: `<a class="nav-link" href="/vinhos">Vinhos</a>`
- SELECTOR: `ul.nav-links > li.nav-item > a.nav-link`
- XPATH: `//ul[@class='nav-links']/li/a`
- Estratégia: CSS combinando hierarquia e classes semânticas.
- Justificativa: A estrutura de lista com classes bem definidas permite selecionar com precisão todos os links de categorias. A hierarquia clara entre `ul > li > a` torna o seletor confiável e reutilizável mesmo que o número de categorias mude.

2º Elemento: Array de Produtos (Cards)



- Nome do elemento: Card de Produto.
- HTML Relevante:

```
<div class="product-card" data-sku="12345">  
  <a href="/produto/vinho-xpto" class="product-link">  
    <h3 class="product-name">Vinho XPTO</h3>  
  </a>  
</div>
```

- **ELEMENT:** `<div class="product-card" data-sku="12345">...</div>`
- **SELECTOR:** `div.product-card`
- **XPATH:** `//div[contains(@class, 'product-card')]`
- **Estratégia:** Classe semântica com hierarquia aplicada em conjunto
- **Justificativa:** A classe `product-card` é aplicada a todos os itens da listagem de produtos, permitindo capturar o conjunto completo. Pode ser refinada com seletores filhos, caso seja necessário mapear nome, imagem ou botão individual.

3º Elemento: Ícone de Lupa da Busca



- **Nome do elemento:** Botão de abertura da barra de busca.
- **HTML Relevante:**

```
<button class="search-toggle" aria-label="Abrir busca">
  <svg class="icon-search"></svg>
</button>
```

- **ELEMENT:** `<svg class="icon-search"></svg>`
- **SELECTOR:** `button.search-toggle[aria-label="Abrir busca"] > svg.icon-search`
- **XPATH:** `//button[contains(@class, 'search-toggle')]/*[name()='svg']`
- **Estratégia:** Atributo + classe semântica + estrutura SVG
- **Justificativa:** A combinação do `aria-label` com uma classe funcional (`search-toggle`) torna o seletor claro, acessível e funcional. O atributo é usado tanto para acessibilidade quanto para automação, enquanto o uso de SVG exige que o seletor contemple filhos.

4º Elemento: Botão "Ir para o Carrinho"



- **Nome do elemento:** Link para página de checkout.
- **HTML Relevante:**

```
<a id="cart-checkout-link" href="/carrinho" class="header-cart-button">
  Ir para o carrinho
</a>
```

- **ELEMENT:** `<a id="cart-checkout-link" href="/carrinho">Ir para o carrinho</a>`
- **SELECTOR:** `a#cart-checkout-link`
- **XPATH:** `//a[@id='cart-checkout-link']`
- **Estratégia:** ID direto
- **Justificativa:** O `id` oferece um seletor simples, único e de alta estabilidade, portanto o `id="cart-checkout-link"` é a melhor opção para interação direta, enquanto seletores baseados em classe (como `checkout-button`) ou texto são mais propensas a mudanças.

Considerações Finais

- As classes CSS da Divvino são bem nomeadas, o que favorece boas práticas de automação.
- Arrays de elementos devem ser sempre capturados por seletores que reflitam sua estrutura hierárquica.

- Quando disponíveis, atributos como `id`, `aria-label` ou `data-*` devem ser priorizados por sua estabilidade.
- XPath é útil para casos específicos (ex: SVGs), mas preferencialmente evitado quando um CSS robusto já atende.